NeuroLab

A neural network built from scratch in NumPy, with an interactive optimizer playground

Neural Networks and Deep Learning: course project report

Dataset: UCI Energy Efficiency (id 242) and generated 2-D classification sets

Team member A (model lead): [name and roll number]

Team member B (application lead): [name and roll number]

Repository: [Git repository URL]

Deployed app / demo video: [URL]

Contents

1. Introduction
2. Background
3. Implementation
4. Verification by numerical gradient checking
5. Experiments
6. Web application
7. Limitations and future work
8. Conclusion
9. Individual contributions
10. References

1. Introduction

Most students meet neural networks through a single call such as model.fit(). The optimisation, the chain rule and the choice of activation function stay hidden. NeuroLab removes that black box. We wrote a small neural network library using only NumPy, proved numerically that its gradients are correct, rebuilt the same network in PyTorch as a cross-check, and put everything behind a web playground where a user can change the architecture, activation, optimizer and learning rate and watch learning happen live.

The project has five goals, taken from the assignment:

- Implement Dense layers, five activations (sigmoid, tanh, ReLU, Leaky ReLU, softmax) and two losses (MSE, cross-entropy) with hand-written forward and backward passes.
- Implement four optimizers by hand: SGD, SGD with momentum, RMSprop and Adam.
- Show by numerical gradient checking that every layer's gradient has relative error below 1e-5.
- Train a multi-output regressor on UCI Energy Efficiency, rebuild it in PyTorch, and compare test error. The success target is a heating-load test R^2 of at least 0.95.
- Compare SGD, RMSprop and Adam on the Rosenbrock and Beale surfaces, and write a comparison of convergence speed.

Team roles followed the brief. Member A built the data pipeline, the library and the experiments. Member B built the FastAPI service, the web interface, prediction logging and deployment. Both members co-wrote this report.

2. Background

2.1 Biological and artificial neurons

A biological neuron receives signals on its dendrites, sums them in the cell body and, if the total is large enough, sends a spike down the axon to other cells through synapses. An artificial neuron keeps only this weighted-sum-then-threshold idea. Figures 1 and 2 are our own diagrams, and the same explanation is available in the web app on the /neurons page.

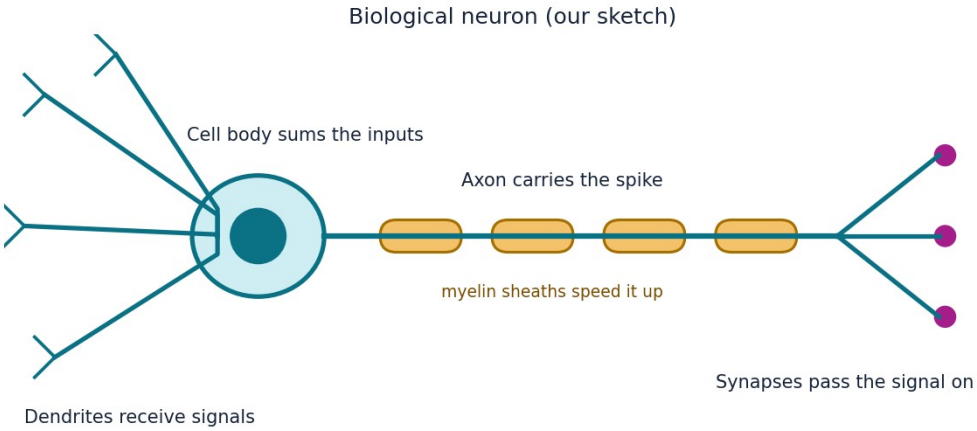


Figure 1. Biological neuron (own sketch).

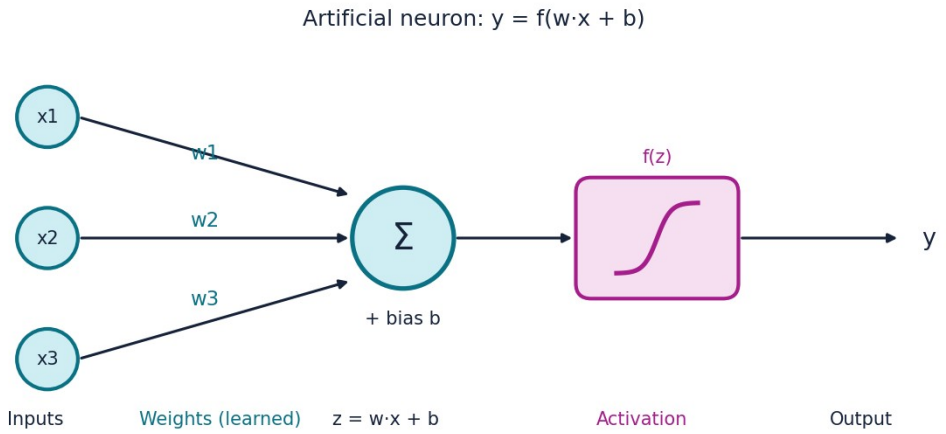


Figure 2. Artificial neuron: y = f(w·x + b).

Part	Biological neuron	Artificial neuron
Inputs	Chemical signals at the dendrites	Feature vector x
Connection strength	Synapse strength, changed by experience	Weights W, changed by gradient descent
Combining inputs	Cell body integrates incoming charge	z = xW + b
Firing rule	Spike above a membrane-potential threshold	Activation function f(z)
Learning	Synaptic plasticity	Backpropagation and an optimizer

2.2 Forward pass and activations

A Dense layer computes z = xW + b for a batch x of shape (N, in), W of shape (in, out) and b of shape (1, out). A non-linear activation follows each hidden layer. Without it, a stack of layers is one linear map and cannot bend a decision boundary. Table 1 lists the activations and derivatives that we implemented and verified.

Activation	Forward	Backward (given dL/dy)
Sigmoid	y = 1 / (1 + e^-x)	dx = dy · y(1 − y)
Tanh	y = tanh(x)	dx = dy · (1 − y^2)
ReLU	y = max(0, x)	dx = dy · [x > 0]
Leaky ReLU	y = x if x > 0 else 0.01x	dx = dy · (1 if x > 0 else 0.01)
Softmax (row-wise)	y_i = e^(x_i − max) / Σ e^(x_j − max)	dx = y · (dy − Σ(dy · y))

Table 1. Activations. The max is subtracted inside softmax so that exp never overflows.

2.3 Losses

$$\text{MSE:}\quad L = \text{mean}\left((\hat{y} - y)^2\right), \qquad dL/d\hat{y} = 2(\hat{y} - y) / (N \cdot D)$$

$$\text{Cross-entropy:}\quad L = -(1/N) \sum \sum t_nc \log p_nc, \qquad dL/dp_nc = -t_nc / (N \cdot p_nc)$$

Both losses average over the batch. That is why the Dense layer's backward pass needs no extra 1/N factor.

2.4 Backpropagation for one Dense layer

Let g = dL/dy be the gradient arriving from the layer above. By the chain rule applied to y = xW + b:

$$dW = x^T g, \qquad db = sum\ of\ g\ over\ the\ batch\ rows, \qquad dx = g\ W^T$$

The layer returns dx so that the layer below can continue the chain. The network calls backward() on the loss first, then on every layer in reverse order.

2.5 Optimizers

Optimizer	Update rule	Main idea
SGD	$\theta \leftarrow \theta - \eta\, g$	One global step size
Momentum	$v \leftarrow \mu\, v - \eta\, g$; $\theta \leftarrow \theta + v$	Velocity smooths noise
RMSprop	$c \leftarrow \rho\, c + (1-\rho)\, g^2$; $\theta \leftarrow \theta - \eta\, g / (\sqrt{c} + \varepsilon)$	Per-weight step size
Adam	$m \leftarrow \beta_1 m + (1-\beta_1) g$; $v \leftarrow \beta_2 v + (1-\beta_2) g^2$; $\theta \leftarrow \theta - \eta\, (m/(1-\beta_1^t)) / (\sqrt{(v/(1-\beta_2^t))} + \varepsilon)$	Momentum plus RMSprop, with bias correction

Table 2. Optimizers implemented by hand ($\mu = 0.9$, $\rho = 0.9$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 1e\text{-}8$).

3. Implementation

The library lives in the neurolab package. It uses NumPy only; there is no autograd anywhere. Each component has one job:

File	Contents
activations.py	Identity, Sigmoid, Tanh, ReLU, LeakyReLU, Softmax, each with forward and backward
layers.py	Dense layer with He or Xavier initialisation and manual gradients dW, db, dx
losses.py	MSE and CrossEntropy with backward()
optimizers.py	SGD, Momentum, RMSprop, Adam operating in place on lists of arrays
network.py	Network container, build_mlp(), mini-batch training generator fit_iter(), R^2
gradcheck.py	Central-difference gradient checker for activations, losses and whole networks
data.py, energy.py	Energy Efficiency loading, fixed-seed split, standardisation, training and model saving
torch_baseline.py	The same network in PyTorch, with optional copying of the NumPy initial weights
playground.py	Streaming training engine used by the web app

Design decisions that matter for correctness:

- He initialisation for ReLU-like layers and Xavier initialisation for tanh, sigmoid and the output layer, so activations neither vanish nor explode at the start.
- Optimizers update arrays in place and know nothing about networks, so the identical code trains the neural network and walks the 2-D test surfaces.

- Batch order depends only on the seed. Two runs with the same seed and cloned initial weights therefore differ only by optimizer, which is what a fair comparison needs.
- Every random component uses a fixed seed, and the standardiser is fitted on the training split only.

4. Verification by numerical gradient checking

For each parameter tensor θ we compare the analytic gradient from backward() with a central finite difference (L(θ+h) − L(θ−h)) / 2h with h = 1e-6. The reported number is the relative error ||a − n|| / (||a|| + ||n||). The assignment requires this to be below 1e-5 for every layer.

The check covers six activations, both losses, and 36 weight and bias tensors from full networks (four hidden activations with a regression head, and tanh and ReLU with a classification head). Table 3 shows the worst error in each group.

Group	Checks	Worst relative error	Result
Activations	6	3.5e-10	Pass
Losses (MSE, cross-entropy)	2	3.2e-10	Pass
Network layers (W and b)	36	8.3e-09	Pass
All	44	8.3e-09	Pass, limit 1e-5

Table 3. Gradient check summary (results/gradient_check.json). The worst error is about 1200 times smaller than the limit.

The same check runs from the command line, from the unit tests, and from the Gradient check tab of the web application.

5. Experiments

5.1 Classification on generated data

To see the effect of activations and optimizers directly, we trained a network with two hidden layers of 16 neurons on moons, circles and spirals (400 points each, 25% held out for validation). Figure 3 shows the boundaries learned by Adam. Both tanh and ReLU solve circles and spirals; the ReLU boundaries are visibly piecewise linear, while the tanh boundaries are smoother. On moons both reach 97% validation accuracy.

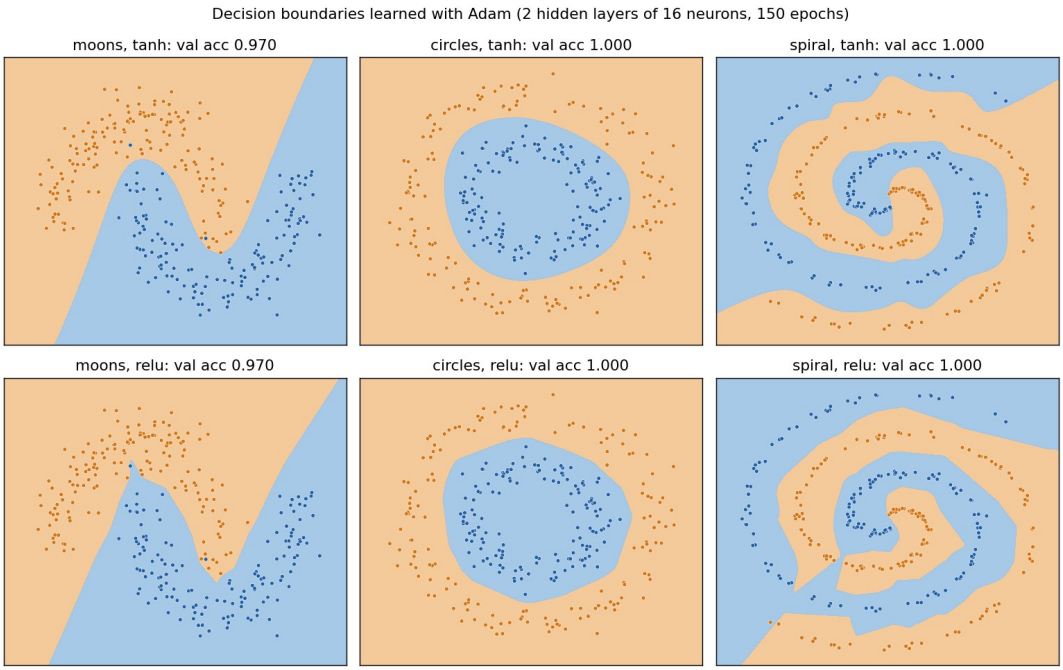


Figure 3. Decision boundaries, Adam, tanh (top) and ReLU (bottom).

Figure 4 compares the four optimizers from identical initial weights and batch order. The learning rates were not tuned per dataset (SGD 0.1, momentum 0.03, RMSprop 0.005, Adam 0.01).

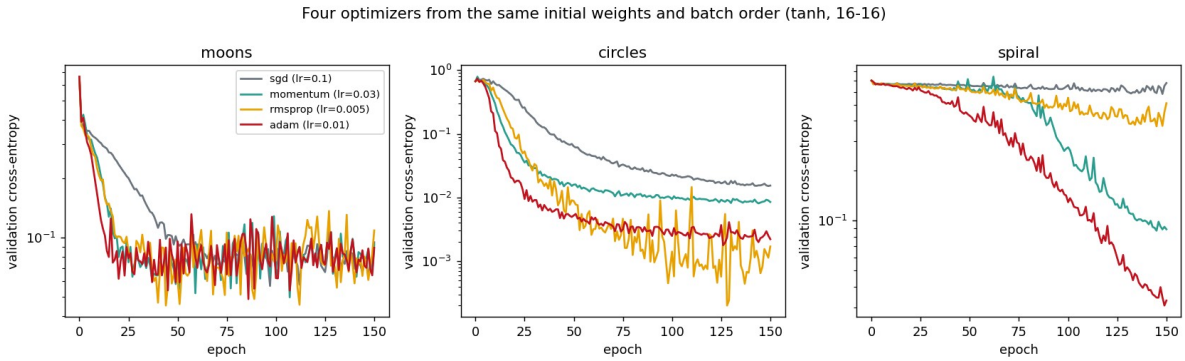


Figure 4. Validation cross-entropy for four optimizers on moons, circles and spiral.

Dataset	Optimizer	Epochs to val loss < 0.2	Final val loss	Val accuracy
Moons	SGD	26	0.095	0.96
	Momentum	12	0.092	0.97
	RMSprop	11	0.109	0.97
	Adam	8	0.089	0.97
Circles	SGD	28	0.015	1.00
	Momentum	12	0.008	1.00
	RMSprop	16	0.002	1.00
	Adam	10	0.002	1.00
Spiral	SGD	not reached	0.677	0.71

Dataset	Optimizer	Epochs to val loss < 0.2	Final val loss	Val accuracy
	Momentum	107	0.089	0.97
	RMSprop	not reached	0.511	0.79
	Adam	84	0.033	1.00

Table 4. Optimizer comparison on the classification sets (results/classification_demos.json), 150 epochs.

The differences are small on the easy sets and large on the spiral. Adam reached the loss threshold first on moons and circles among the four (8 and 10 epochs), and on the spiral only Adam and momentum got there within 150 epochs. With these untuned learning rates plain SGD and RMSprop had not finished learning the spiral by epoch 150. This is one seed and one learning rate per optimizer, so we treat it as an illustration and not as a ranking.

5.2 Multi-output regression on UCI Energy Efficiency

The dataset has 768 simulated building designs, eight inputs (relative compactness, surface area, wall area, roof area, overall height, orientation, glazing area, glazing area distribution) and two targets, heating load and cooling load. We load it from a local ENB2012_data file or with fetch_ucirepo(id=242).

Setup. The data is split 70/15/15 into train, validation and test with seed 42. Inputs and targets are standardised using training statistics only. The network has two hidden layers of 64 ReLU units and two linear outputs, trained with Adam (learning rate 2e-3, batch size 32, 300 epochs) on MSE. The epoch with the lowest validation loss is kept. The test set is evaluated once per model at the end.

Results. [Run python -m experiments.train_energy on the real dataset and copy the numbers from results/energy_results.json into Table 5. Confirm dataset_source is not SYNTHETIC-FALLBACK.]

Model	Target	Test R^2	Test RMSE	Best epoch
NumPy (from scratch)	Heating load	[R^2]	[RMSE]	[epoch]
NumPy (from scratch)	Cooling load	[R^2]	[RMSE]	[epoch]
PyTorch, same initial weights	Heating load	[R^2]	[RMSE]	[epoch]
PyTorch, same initial weights	Cooling load	[R^2]	[RMSE]	[epoch]
PyTorch, default init	Heating load	[R^2]	[RMSE]	[epoch]
PyTorch, default init	Cooling load	[R^2]	[RMSE]	[epoch]

Table 5. Energy Efficiency test results (RMSE in kWh/m²).

Discussion. [State whether heating-load test R^2 reached 0.95. Say how close the NumPy and PyTorch curves are (results/energy_loss_curves.png) and explain why: with copied initial weights, the same batch order, float64 and identical Adam settings, both should follow almost the same trajectory. Comment on the default-init run and on which target is harder.]

Figures to insert: [energy_loss_curves.png and energy_pred_vs_actual.png from results/]

5.3 Optimizer paths on Rosenbrock and Beale

We ran each optimizer for 3000 steps from a fixed start point: (−1.5, 2) on Rosenbrock, whose minimum is at (1, 1), and (−1, −1.5) on Beale, whose minimum is at (3, 0.5). Learning rates were set once per surface so that each optimizer is reasonably stable; they are listed in Table 6.

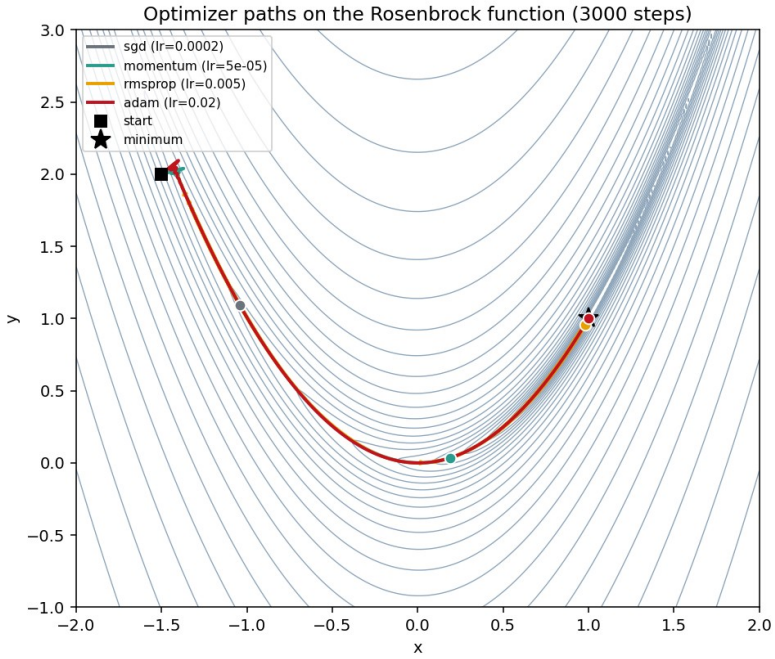


Figure 5. Optimizer paths on the Rosenbrock function.

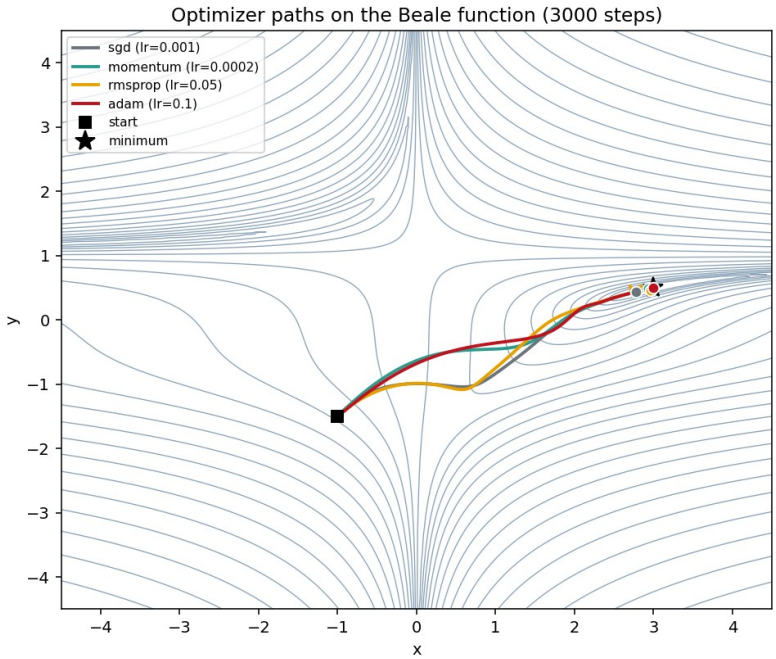


Figure 6. Optimizer paths on the Beale function.

Surface	Optimizer	Learning rate	Final f	Distance to minimum	Steps to f < 0.01
Rosenbrock	SGD	2e-4	4.18	2.04	never
	Momentum	5e-5	0.656	1.26	never

Surface	Optimizer	Learning rate	Final f	Distance to minimum	Steps to f < 0.01
	RMSprop	5e-3	5.9e-3	0.049	1860
	Adam	2e-2	3.1e-11	1.3e-5	1247
Beale	SGD	1e-3	9.99e-3	0.228	2999
	Momentum	2e-4	7.6e-4	0.069	1393
	RMSprop	5e-2	2.3e-2	0.055	128
	Adam	1e-1	5.1e-30	~0	281

Table 6. Results after 3000 steps (results/optimizer_paths_summary.json).

Rosenbrock has a long, narrow, curved valley. The gradient points mostly across the valley, so a step size that is safe for the steep walls makes very slow progress along the floor. SGD and momentum did not get close within 3000 steps with the small learning rates the walls require. RMSprop and Adam rescale each coordinate by its recent gradient size, which lets them move along the valley, and Adam converged to the minimum. On Beale, Adam converged exactly, RMSprop was the first to get below f = 0.01 (128 steps) but did not stay there: its final value was 0.023, which suggests it was still jittering around the minimum at its constant step size. Momentum and SGD arrived more slowly. The learning rates differ per optimizer, so these numbers show behaviour under one reasonable setting, not a proof that one method is always better.

5.4 Convergence speed of the four optimizers on Energy Efficiency

Following the assignment, we also compare convergence speed on the regression task. Every optimizer starts from the same cloned weights, sees the same mini-batch order, and receives the best learning rate from a small grid chosen by validation loss. The measure is the number of epochs until validation MSE (standardised units) first falls below 0.05, which is roughly $R^2 = 0.95$.

[Run python -m experiments.compare_optimizers and paste the table from results/convergence_comparison.md here, together with results/optimizer_convergence.png.]

Optimizer	Best lr	Epochs to val MSE < 0.05	Epochs to < 0.01	Best val MSE
SGD	[lr]	[epochs]	[epochs]	[MSE]
Momentum	[lr]	[epochs]	[epochs]	[MSE]
RMSprop	[lr]	[epochs]	[epochs]	[MSE]
Adam	[lr]	[epochs]	[epochs]	[MSE]

Table 7. Convergence speed on Energy Efficiency.

Written comparison. [Describe in your own words which optimizer converged fastest and which was most sensitive to the learning rate, using Table 7 and the grid in convergence_comparison.json. Relate it to the mechanisms in Section 2.5 and to the behaviour on the classification sets (Section 5.1) and the test surfaces (Section 5.3). If your result differs from what theory suggests, explain why.]

6. Web application

The application has a FastAPI back end that serves the NumPy model and a plain HTML, CSS and JavaScript front end. The model is always served behind our own API; no external model service is called.

Endpoint	Purpose
POST /api/train/stream	Streams a live training run as newline-delimited JSON: one init message, then one epoch message per run per epoch, then done
GET /api/energy/info	Metadata and test metrics of the served Energy model
POST /api/predict	Heating and cooling load for one building design; validates ranges and appends every request to logs/predictions.jsonl
GET /api/gradcheck	Runs the numerical gradient check and returns the table
GET /api/health	Liveness check

Playground tab. The user chooses the dataset (moons, circles, spiral or Energy Efficiency), the number of hidden layers and neurons, the activation, the optimizer and the learning rate. While the server trains, the browser draws the loss curve and either the decision boundary (classification) or a predicted-versus-actual scatter plot (regression). A second optimizer can be enabled to run side by side. The server clones the initial network and reuses the same batch order, so the two runs differ only by optimizer. If a learning rate makes the loss diverge, that run stops and the interface says so.

Energy predictor tab. Eight input fields take a building design and return predicted heating and cooling load, together with the model's held-out test metrics. The page warns if the served model was trained on the offline stand-in data.

Gradient check tab and neuron page. The gradient check can be run from the browser, and the /neurons page contains the biological-versus-artificial comparison with our own diagrams.

Deployment. [Describe where the app is deployed (for example a Hugging Face Space) and give the URL, or state that a recorded demo video is provided and give its link.]

The repository includes a Dockerfile that listens on port 7860, which suits Hugging Face Spaces and also works on Render.

7. Limitations and future work

- The Energy Efficiency dataset is small (768 rows) and simulated, so high R^2 does not guarantee accuracy on real buildings.
- The optimizer comparisons use one seed and hand-picked or grid-picked learning rates. Repeating over several seeds and reporting mean and spread would make the conclusions stronger.
- Only fully connected networks are implemented. Convolutional layers, dropout, weight decay and batch normalisation are natural extensions.
- The playground trains on the server; many simultaneous users would need a job queue or in-browser training.
- ReLU gradients are not defined exactly at zero, so the gradient check can be noisy if a pre-activation lands very close to zero.

8. Conclusion

We built a working neural network library without autograd, verified its gradients to better than 1e-8 relative error on all 44 checks, and used it to study activations and optimizers on generated classification data and on standard test surfaces. The web playground lets anyone repeat those experiments interactively, including a fair side-by-side comparison from identical starting weights. The Energy Efficiency results and the final optimizer ranking on that dataset are reported in Sections 5.2 and 5.4.

9. Individual contributions

Member	Contribution
[Member A name]	[Data pipeline, activations, layers, losses, optimizers, gradient check, experiments, ablation tables]
[Member B name]	[FastAPI service, web interface, prediction logging, deployment, demo video]
Both	Report writing, code review, viva preparation. Commit history from both members is in the repository.

10. References

- A. Tsanas and A. Xifara, “Accurate quantitative estimation of energy performance of residential buildings using statistical machine learning tools,” Energy and Buildings, vol. 49, pp. 560–567, 2012.
- D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” ICLR, 2015.
- T. Tieleman and G. Hinton, “Lecture 6.5, RMSprop,” COURSERA: Neural Networks for Machine Learning, 2012.
- D. E. Rumelhart, G. E. Hinton and R. J. Williams, “Learning representations by back-propagating errors,” Nature, vol. 323, pp. 533–536, 1986.
- X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” AISTATS, 2010.
- K. He, X. Zhang, S. Ren and J. Sun, “Delving deep into rectifiers,” ICCV, 2015.